Pre-read

CASE 4 · YOUR OWN CHIEF OF STAFF

MESA AI and its Applications · For Session: Friday, 10th October 2026

THE PREMISE

For three cases you sat in someone else's seat. Meera's, Arjun's, Nikhil's. You built tools for their problems, in their businesses, using inputs they gave you.

This case is different. The principal is you. The problem is yours. The inputs are the ones sitting in your MESA Gmail, your Drive, and your Google Calendar right now - the pre-reads you marked unread, the deadlines you are tracking in your head, the emails you have been meaning to reply to for three days.

You are about to build a Chief of Staff. Not a chatbot. A system that retrieves, drafts, cleans, schedules, and stops - and that you will still be using in March.

THE SITUATION

You are managing a 12-month intensive programme across multiple courses, terms, and instructors. Every week, Pre-reads live on Nexus - you read them there or download and save it to your MESA GDrive as PDFs. Deadlines are communicated by email, confirmed verbally in class, and sometimes updated on Nexus. Your course material accumulates in Drive - case briefs, session notes, execution plans - unorganised and unsearchable. By Week 4 of any term, you spend more time locating what you already have than working with it.

None of this is a knowledge problem. You have access to everything you need. The problem is that the information is scattered across four places - Gmail, Drive, Calendar, and memory - and you reconstruct the full picture from scratch every week. Nothing from Case 1 made Case 2 easier. Nothing from Term 1 will make Term 2 easier, unless you build something that compounds.

In ten months you will be an AI generalist looking for work, or doing it. The job has a shape you already recognise: someone hands you an ambiguous problem, you find out what is actually true, and you come back with something specific, sourced, and honest about what you could not establish. A Chief of Staff does exactly that. The difference between you and someone who cannot say they built this is whether you built it before you were asked to.

Six tasks. One Telegram interface. Everything running on your MESA ID - the same Gmail, Drive, and Calendar you already use every day.

THE NUMBERS

- 6 tasks behind one Telegram interface, routed by a single classifier, assessed individually
- 3 integrations - your MESA Gmail, Google Drive, and Google Calendar - one OAuth consent screen covering all three

- 3 correct cited answers + 3 clean refusals - the only passing threshold for Task 1. Not 3 answers. Not 3 refusals. Both.
- Every answer carries its source - document name and the specific location within it. No citation means no answer.
- 1 line that matters above all others: *"Not found in the documents provided."* Not a hedge. Not a paragraph that reads like an answer. One line, and stop.
- 1 Monday digest, automated at 8am, nothing repeated from the previous week

THE ASK

> *"Six things from my phone. If the answer is in my course material, it finds it and tells me exactly where. If it isn't, it says so and stops. It reads my calendar and briefs me before a class I haven't prepared for. It cleans a pile of documents and flags every row it wasn't sure about. It drafts any email I need to send, shows it to me, and waits. I tell it to block time or add a session to my calendar and it does it, immediately. On Monday morning it tells me what I need to do before Friday. It never sends anything on its own. Not once."*

Your app must also account for what it costs to run. Every call that reaches an LLM is logged - task name, token count, and timestamp - to Supabase. A “/costs” command in Telegram returns a summary of usage and estimated spend, broken down by task. You must be able to tell, at any point, which task is your most expensive and what you have spent today.

SCOPE OF THIS BUILD

IN SCOPE

- **Task 1: Ask My Stuff -** Retrieval from your corpus in Drive, with citations. Demonstrated refusal on three questions that are verifiably not in the corpus. Retrieval uses BM25 alongside vector similarity - both must be working. Both halves of Task 1 are assessed separately: citations and refusal are not the same test.
- **Task 2: Brief Me Before Class -** Reads tomorrow's calendar entries for MESA sessions, finds the relevant pre-read or case material in your corpus, and writes a structured brief to Drive - with a WHAT I COULD NOT ESTABLISH section. Triggered by the calendar, not by you asking.
- **Task 3: Clean This Up -** Extracts structured data from your Drive or Drive folder - PDFs, scanned documents, photographs of paper. Outputs a clean Google Sheet in Drive. Flags every row it was uncertain about. Nothing is guessed; uncertain extractions are surfaced, not silently entered.
- **Task 4: Draft This -** Drafts a reply to any email you forward or describe via Telegram. Shows you the draft. Holds it at the gate. Nothing is sent, ever. Add the gate before you add the drafting.
- **Task 5: Add This to My Calendar -** Creates a Google Calendar event from a plain English Telegram message. "Block 3 hours Thursday afternoon for Case 4 prep" becomes a calendar event, immediately confirmed back to you. Reads the calendar first to avoid conflicts.

- **Task 6: Monday Digest -** Every Monday at 8am: what's due this week from your corpus and Gmail, what's on your calendar, what the system did last week. Nothing repeated from the previous digest. Runs on Vercel Cron.
- **Economics monitoring:** a “/costs” Telegram command that returns token usage and estimated spend per task, calculated from Supabase logs.

OUT OF SCOPE

- Sending any email, message, or form on your behalf - ever. If you add sending "temporarily to test," you will not remove it before the assessment. Build the gate first, always.
- Answering Task 1 questions from general knowledge when the answer is absent from your corpus. That is not a fallback. It is the failure mode the entire task is designed against.
- Adding new integrations beyond Gmail, Drive, and Calendar. Six tasks on three integrations. Scope is the constraint, not a limitation of ambition.

WHAT YOU HAVE ACCESS TO

- **MESA Gmail**: Your live inbox, connected via OAuth. Tasks 2 and 6 read it for upcoming deadlines and unread pre-reads. Task 4 drafts replies to emails you forward or describe.
- **Google Calendar**: Your live calendar, connected via OAuth. Tasks 2, 5, and 6 read it. Task 5 writes to it. Drive and Calendar share one OAuth consent screen - one connection covers both.
- **Google Drive**: All or specific folders in your Google Drive, also connected via OAuth.

YOUR SEAT

You are the operator and the principal. There is no client to hand the build to. The assessment is: does it work when you use it, from your phone, on your real calendar and your real inbox, in real time.

Three decisions to make before you write a single line of code:

- Presence: For Task 1, what does "the answer is genuinely present" mean? Not vaguely related. Not implied by. Directly and specifically answerable from the retrieved passages. You will implement this as a structured prompt with a typed boolean output. Know what you are implementing before you write the prompt.
- Scope: Drive read-only or read-write? Calendar read or read-write? Justify each out loud before you configure the OAuth consent screen. The instinct is to request everything. The correct answer is the narrowest scope that does the job - read-only for Drive and Gmail, read-write for Calendar. Being able to explain that distinction in the assessment write-up is worth more marks than the integration itself.
- Routing: The router classifies every incoming Telegram message into one of six tasks. Test it on ten ambiguous messages before you build anything behind it. A wrong router makes every task feel broken, and you will debug the wrong thing.

BEFORE THE SESSION: 40 MINUTES

1. OAuth setup · *15 min*

A guide goes out with this pre-read. Follow it step by step. Your build is an unverified app - but because this is an internal MESA build, it will work. In the pre-read guide, you will see a standard Google consent prompt to click through. Arrive with credentials in hand - Client ID and Client Secret for Drive and Calendar.

2. Read before you build · *10 min*

Every programming language has a way to borrow code that someone else has already written, tested, and published for anyone to use. In Python, these are called libraries - and they live in public repositories, mostly on GitHub. Using one doesn't mean you didn't build your project. It means you didn't waste time solving a problem that's already been solved. A civil engineer doesn't calculate the tensile strength of steel from first principles every time; they use published material standards. A public library is the same idea applied to code.

Task 1 needs to search your corpus efficiently before it sends anything to the LLM. The naive version - send all 30 documents to the LLM on every query - burns token costs proportional to your corpus size, every single time. The smarter version pre-filters: score the documents first, cheaply, then send only the top-ranked ones. That filtering step is what rank_bm25 does.

Open **dorianbrown/rank_bm25** on GitHub. Read the README - specifically *BM25Okapi initialisation* and the *get_scores() method*. That's the entire read.

3. Six questions · *5 min*

Write down six questions you would actually ask a Chief of Staff this month. Real ones. Mark each one: answerable from my corpus/answerable from my Gmail or Calendar/not answerable by anyone. The third category is not a gap in the build. It is the finding - and three of those unanswerable questions become your Task 1 refusal tests during the session.